A Reactive Infrastructure for Swarm Programming

by Lorenzo Manfredi Segato, Filippo Marcantoni & Emma Pollak
Project Advisor: Carlo Pinciroli

Abstract

We present PiELo, a novel programming language for swarm robotics. A robot swarm is a decentralized network of robots working to achieve a common goal. Robot swarms have a multitude of useful applications, including exploration, construction, and mining. When programming swarms, the robots need to respond to changes in the environment. If a sensor reading updates, the robots should respond to the new readings and take action if necessary. This process, known as reactivity, is the essential building block for robots to take appropriate actions and contribute to the swarm's overall goals. Additionally, in a problem unique to swarms, robots need to be able to form a consensus, i.e., they must reach an agreement on important aspects of their environment. Programming these behaviors currently requires a significant effort and expertise from the programmer. PiELo explores addressing these concepts as first-class features of a programming language. When programming in PiELo, reactive primitives enable the programmer to easily write functions that will stay up-to-date as the robot performs its duties. Additionally, PiELo provides multiple means through which individual robots can come to a decision and compute problems together. This enables many foundational swarm algorithms, such as flocking and gradient formation. In our paper, we show the effectiveness of this new language through a series of demonstrations of common swarm algorithms and problems, comparing the required code in PiELo to the same algorithms written in other contemporary robotics languages.

Chapter 1. Introduction

Swarm intelligence is a scientific discipline that studies the design of distributed systems or algorithms inspired by the collective behavior of social insects and other animal societies [1]. In the robotics paradigm, this idea is applied to enterprises that require autonomous behaviors, specifically those which are scalable and reliable [2]. In essence, swarm behaviors are made up of small, local interactions that build up and spread through the entities in the swarm leading to some form of global behavior (swarm-level goals) [3]. Yet in practice, building real‑world swarm robots remains difficult: coordination, communication deadlines, physical interference, and scale (many robots in large environments) all conspire to make implementations brittle and labor‑intensive [4].

In order to bridge this gap, there are two main families of approach to design robotic swarm systems: Automatic and "manual" methods: For the former, the individual behavior of robots is treated as a "black box", characterised by a set of parameters. An example of this could be a neural network implemented for evolutionary robotics [4]. A general method for this approach is to search for parameter values through optimization techniques. Following from the evolutionary robotics example, this can take place through a genetic algorithm. From there, the quality of the solution is measured (perhaps through statistical processes) to direct the search for better values, typically achieved through physics-based simulations in software such as ARGoS [5]. This technique, while not requiring much design experience, proves itself difficult to analyze and poorly scalable with marginally added parameters. Manual approaches, on the other hand, required human insight to craft some candidate behaviors, which are then prototyped and run through simulations (equivalent to those in the automatic method). This cycle is repeated until the simulations are satisfactory, leading to in-robot implementation (a challenge of its own merit, discussed in more detail hereafter) until, finally, the implementation is successful and functional. With this approach, despite gaining deeper knowledge about the problem and the solution, there is no real methodology involved; as such, to navigate the challenges presented by a lack of procedure, a large amount of design experience is required.

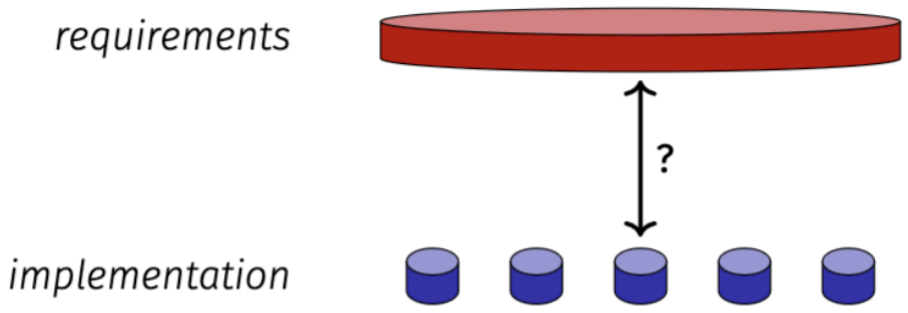


Figure 1: Diagram showcasing the gap in the requirements needed for swarm robotics and the current implementation methods

Beneath these design workflows lies the question of swarm programming: How do we write software that runs on each robot and collectively yields reliable, emergent behaviors? To understand this, we must first understand the computation of swarm machines. In short, computation - in the general sense - can be thought of as a function that takes in a state and changes this state based on this function (see Fig. 2 below) [6].

$$f : \mathbb{S} \to \mathbb{S}$$

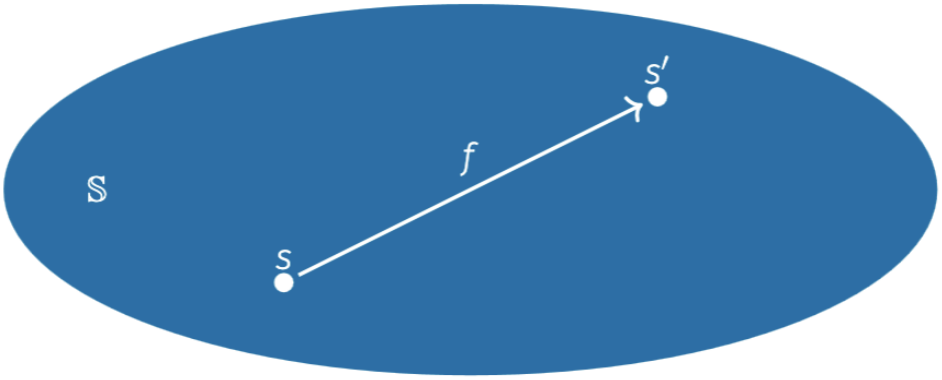


Figure 2: Diagram illustrating computation [6]

In the case of swarm robotics, it is paramount to clearly identify what this state is, due to its inherently high sophistication. This sophistication, in no small part, is derived from the fact that the robots are exposed to and in constant communication with a partially unknown and unobservable environment (the real world). Adding to this, uniquely to swarms, is the issue of inter-robot communication. While at first it may seem that robots in homogenous swarms are identical to each other - and while in principle they may be - in reality, due to tolerances and environmental exposure, their differences widen leading to communication issues in their own merit but also with respect to concepts such as consensus (discussed more deeply Chapter 3). In summary, the information flow of the state in the a swarm machine can be described through the following diagram, where one must concern themselves with the internal state of each robot (memory, battery levels), the physical state of each robot (mechanics), the communication topology between the member robots of the swarm and the relevant environment state (stigmergy).

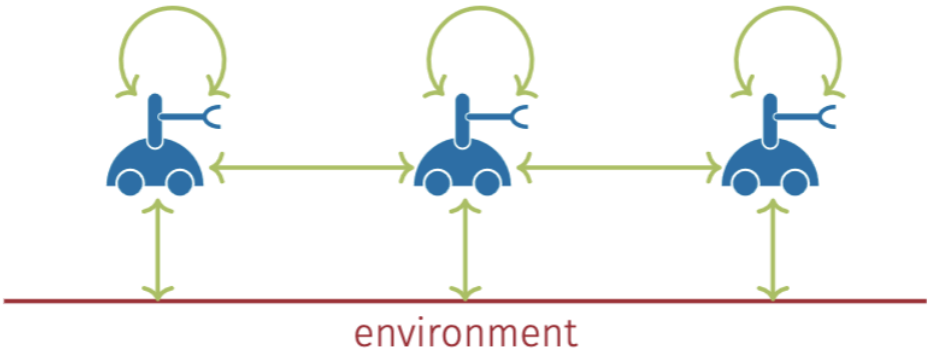


Figure 3: Diagram illustrating the interaction between robot swarms and the environment [6]

An additional challenge in this respect is the fact that a swarm is an “open” machine [6]. By that we mean that the programmer has no inherent control of the state: Not only is the environment dynamic and partially unknown, but so is the state of the swarm itself. In a singular robot, diagnostic procedures can be implemented to provide developers with relevant information regarding the robot’s current state. However, in a swarm, all these intricacies are true for every member robot, making it computationally expensive to attempt to determine these values for every robot, even before attempting to ascertain the state of the swarm itself. Further still lies the problem of communication; not only ensuring that messages between robots are being transmitted and received correctly, but validating the correctness of the content of these messages.

Parallels can be drawn between the issues we have described and those observed in classical distributed systems. However, it is notable that the topology in distributed systems can be controlled and made static - a “closed” machine where only the system itself, governed by reliable high-bandwidth communication experiencing rare failures, can affect its state [6]. One could also liken swarms to mobile sensor networks, but their focus is usually more skewed towards data collection and aggregation of the environment - an “ajar” machine: while one may not have control over the environment itself, the methods of data collection and aggregation are to the developer’s discretion. In this respect, swarms also tend to modify their environments directly, creating the problems discussed later.

Here we arrive at the programming paradigm for swarms: Based on the issues described, we look for features such as composability, predictability and the support for heterogeneous swarms [6]. Composability refers to the ability of taking a large problem breaking it into smaller abstractions (be it functions, modules, or classes) and the interactions between these abstractions. Predictability refers to the idea that running the same program multiple times yields the same result (given some tolerance). Finally, we look for features that permit that robots in the swarm perform desired behaviors (move, use sensors, actuate, communicate).

An important note about those desired robot behaviors is their event-driven nature. Robots are naturally event-driven systems: sensors produce events, which the robot responds to with some thought and potentially action. In a swarm, this goes doubly, as other robots in the swarm can generate events that require response. This property of robots lends itself well to a programming paradigm known as reactive programming. In reactive programming, relationships are set up between variables to automatically recompute them whenever necessary. This framework allows the programmer to easily respond to events as the recomputation flow matches the conceptual flow of responses to a stimulus. This is a highly desirable feature as it simplifies many common programming patterns.

Recent work in distributed and reactive programming offers key insights for designing distributed systems that are robust to partial failures, operate under weak connectivity, and maintain consistency without centralized coordination. Asynchronous Functional Reactive Programming, exemplified by Elm, shows how systems can remain responsive to local events without the need for global order. In parallel, Conflict-Free Replicated Data Types (CRDTs) provide a model for maintaining eventual consistency across decentralized agents, even with poor connectivity or message delays. Merkle-CRDTs enhance this by using graph-based causal tracking to efficiently synchronize state across peers, by sharing only the branches that other replicas are missing which enhances communication significantly. Although these paradigms are not designed specifically for swarms, they have a lot of potential to enable resilient, coordination-free behaviors in swarm systems.

Currently, there are a few programming paradigms for robotics who try to make use of some of these desired features. A robot-oriented paradigm is the popular Robot Operating System, ROS. A distributed system under the observer pattern following a publish/subscribe architecture, ROS provides a communication interface for users, allowing for the creation of nodes that fulfill requirements specific to the task at hand. However, ROS's focus (even in new releases such as ROS2) is in individual robots. As such, even though there are several packages based on ROS2 for swarms, ROS2 requires a lot of overhead and setup to support multiple robots. For example, the configuration and maintenance of namespaces, which forces many low-level details on the programmer. These low-level details, such as topic management and memory management (if using C/C++), can be incredibly cumbersome, as the programmer is forced to implement and manage every detail themselves. Additionally, the deployment of new software is done manually. This means that there is poor scalability for multiple robots, making it incredibly cumbersome to use for large swarms. Another model is the spatial computing paradigm, where the swarm is thought of as a spatially continuous entity, each point of the swarm is associated to a tuple. This is an appropriate abstraction for simple space-time computations, but the concept of a "robot" is lost - there is no explicit support for actuation or heterogeneity. An example of this paradigm is Protelis [7]. A task-oriented paradigm is one in which robots are controlled and abstracted through tasks that must be completed. One must specify what to do for every task and scheduling is transparent, as observed in tools such as Karma and Voltron [8]. The main drawback of this approach is the assumption of the existence of an effective task scheduler, which is in itself a significant research problem. Finally, researchers have explored goal-oriented paradigms, in which the language specifies the goal state of the swarm, leaving the logic required to achieve the goal to the runtime. An example of this class of languages is SWARMORPH [9].

Many of these programming paradigms offer pieces of the puzzle (modularity, distribution, or goal-oriented APIs) but none provide a concise, event-driven model that natively captures reactive behaviors and distributes them automatically across a swarm. Reactive event-driven programming, where robots react to events (such as sensors updating) without boilerplate consensus-achieving mechanisms (such as message handling), is what swarm developers need - but there are little software tools that are made specifically for the challenges unique to swarms [10]. Here lies the scope of our project: Create a domain-specific language and runtime platform for swarm robotics whose central goal is to make it easy to program event-driven reactive behaviors at the swarm scale.

To achieve this goal, we have created PiELo, a DSL which provides (i) first-class reactive variables: these make it so that an assignment to a variable can be declared as reactive. This assignment creates a dependency (this variable depends on the value of other variables). This is helpful as it allows for complex calculations and functions to be instantly re-calculated throughout the swarm, without having to write complex callback functions under a message-passing paradigm, making the implementation of swarm algorithms much easier and faster; (ii) automatic variable distribution: shared reactive variables are synchronized through a shared variables data structure. A change on one robot is propagated to all others, triggering local reactions (iii) concise DSL syntax: we have developed a Lisp-inspired syntax that blends imperative control (loops, conditionals) with declarative reactivity, all compiled to compact bytecode executed by a simple VM on each robot; (iv) extensibility via native functions; with PiELo, we allow for easy registration of C routines for sensors, actuators or computation, enabling PiELo to interface with real hardware without compromising the reactive model we have developed.

Chapter 2. Related Work

The challenges of swarm programming arise from the complexity of coordinating distributed autonomous agents to achieve collective goals. Several programming paradigms and frameworks have been proposed to address these issues, typically focusing on either bottom-up or top-down approaches. Traditional programming languages for robotics often rely on bottom-up approaches, such as the Robot Operating System (ROS), where each agent is programmed individually to contribute to the overall task. Swarm robotics, on the other hand, requires novel approaches capable of handling decentralized coordination and supporting emergent collective behaviors.

Historically, bottom-up approaches have dominated programming languages. A notable example is SWARMORPH-script [9], a behavior-based scripting language designed for morphogenetic behaviors consisting of forming coherent shapes with mobile robots. More recently, top-down methodologies have gained interest, particularly in the sensor network and spatial computing communities. Languages such as Proto [11], developed in the spatial computing community, approach distributed systems as a continuous spatial medium where each device in the network operates on data tuples, producing predictable behaviors. Despite its modular design and spatial abstraction features, Proto suffers from limitations in robotics applications, including poor support for motion, state persistence, and heterogeneous systems. To overcome these limitations, Protelis [7] was introduced as a Java-based extension that provides Proto's abstraction while making it easier to integrate with existing robotics frameworks and enabling real-world applications.

Similarly, Meld [12] implements a top-down approach by enabling developers to define high-level swarm goals though logic rules and facts. These are progressively evaluated to drive the swarm behavior, abstracting away the low-level coordination mechanisms from the developers. While Meld supports heterogeneous robots by associating facts with specific capabilities, its rule-based execution can lead to unpredictable behaviors and debugging challenges, limiting modularity and scalability in complex scenarios.

Voltron [8] presents another approach intended for distributed mobile sensing. It allows the developers to define logic across spatial locations, relying on a shared tuple space to handle coordination, either through centralized control, or decentralized control, which is based on the concept of virtual synchrony. Although Voltron’s abstraction simplifies development, it lacks the low-level fine-grained coordination necessary in heterogeneous swarms, limiting its applicability in multi-task scenarios.

PaROS [13] offers a significant contribution by introducing swarm abstract primitives that simplify the programming process. It provides a user-friendly interface where high-level methods correspond to complex swarm behaviors, enabling efficient swarm control. While validated in simulations and real-world tests, PaROS adopts a centralized communication model, which, while simplifying task coordination, imposes strict communication requirements.

2.1. Consensus Strategies

Achieving consensus in swarm robotics is a fundamental challenge for enabling decentralized coordination across distributed agents toward shared goals. One of the most notable contributions in this domain is introduced in Buzz [14], which is a domain-specific language that bridges individual and swarm-level programming, allowing developers to create reusable swarm behaviors efficiently. A

critical feature of Buzz is virtual stigmergy, inspired by the concept of virtual synchrony in Voltron. Virtual stigmergy is a decentralized mechanism that enables consensus by sharing a virtual data structure across the swarm. This allows robots to asynchronously share and update information, and, as the robot modifies a value in the shared data structure, this modification eventually propagates through the swarm, forming consensus through constant interaction. Complementary to virtual stigmergy is Buzz's emphasis on neighborhood-based operations. Each robot maintains a local neighbors structure, which contains spatial and identity information of nearby robots, and this structure is updated at each time step and enables local data aggregation, pattern formation, and consensus-driven behaviors. This structure also allows direct neighbor communication, allowing robots to exchange key-value pairs for localized consensus. These neighbor interactions complement the global stigmergic model, reinforcing the language's hybrid approach that blends local and global coordination mechanisms for robust swarm behavior. This approach has been shown to be robust, with experiments demonstrating that Buzz maintains consensus even with significant communication losses, such as a 75% packet drop rate. However, while effective in diverse swarm applications, the lack of precise control over heterogeneous robots poses a scalability challenge.

Building on Buzz, ROSBuzz [15] integrates the Robot Operating System (ROS) to enhance multi-platform compatibility and to promote interoperability within HMASs. ROSBuzz enables heterogeneous robots to execute the same high-level control script, while reaching consensus via a barrier-based mechanism. Here agents submit their state to a shared swarm table and proceed only once the number of entries matches the swarm size, ensuring synchronized transitions. Although field tests validated the framework's ability to maintain consensus, even under 90% packet loss, as swarm diversity and size grow, managing this abstraction layer becomes increasingly complex, limiting scalability.

Extending the Buzz's consensus model, Moussa and Beltrame [16] introduce statistical model checking (SMC) with UPPAAL to formally verify consensus strategies in swarm behaviors. Their approach models the virtual stigmergy-based consensus mechanism and introduces task-specific barrier functions. They associate task-specific barrier functions with consensus conditions, such as agreement on a leader or task assignment, and evaluate them through both agent-based and abstract swarm-based models. These two modeling strategies were used to capture both detailed interaction between robots, and to abstract away from individual behaviors for scalability purposes. The model checking process combines the Monte Carlo simulations with timed automata to evaluate consensus robustness. This formalism provides insights into consensus dynamics but faces scalability challenges due to state explosion problems, while both communication unreliability and robot failures were shown to impact consensus success in weak topologies.

In contrast, Chen et al. [17] introduce a unified and time-bounded consensus framework that combines concepts from statistical physics, control theory, and network science. The core of the approach uses the Ising model, a mathematical model from statistical mechanics, to optimize energy and ensure that agents minimize disagreement with their neighbors. To maintain connectivity and facilitate efficient information propagation, a spanning-tree algorithm is used, which restructures the communication graph based on the topology. Moreover, the system incorporates gray prediction, a technique to predict future states from limited data, and the Molloy-Reed criterion, a graph-theoretic condition for network connectivity, to enhance robustness against agent or communication failure. The consensus controller ensures that the swarm reaches consensus within a limited timeframe, regardless of size, making it particularly suitable for real-time applications. Simulation results demonstrated strong performance in dynamic and adversarial environments, however, the model's

reliance on a fully connected network, presents a significant limitation, as if an agent or link fails, the spanning tree can collapse, disrupting the consensus process.

Lastly, Li et al. [18] apply consensus in a practical setting involving heterogeneous marine autonomous surface ships (H-MASSs) for collaborative search and rescue missions. The authors designed a consensus controller to synchronize agents' search and rescue progress rates based on local communication graphs. Coordination is achieved by the controller as it continually adjusts the agents' speeds and trajectories to harmonize their progress toward task goals. While this approach enables coordinated motion and task progression, it heavily depends on reliable communication, which is often compromised in real-world environments, and can disrupt the process and the effectiveness of synchronization and consensus mechanism.

Collectively, these works demonstrate various strategies for achieving consensus in swarm robotics, each balancing trade-offs between robustness, scalability, and applicability to heterogeneous systems. However, one concept that has not received much attention is the role of reactivity in swarm programming. In this context, reactivity refers to the ability of the program to handle events in an expressive and declarative way, embedded directly into the programming language. This is particularly significant for swarm robotics as agents are naturally event-driven systems, where sensors or other peers in the swarm produce events, and other individual robots respond reactively to these local or global changes. This property is essential to handle events in a distributed system, and has a lot of potential to make swarm behaviors, such as synchronization, through simple local rules.

2.2. Reactive Programming

Recent work in distributed programming has explored abstractions that enable scalable, decentralized, and robust systems without relying on strong coordination mechanisms. A foundational concept in this regard is reactivity, the ability of a system to respond dynamically to local or global changes. In traditional Functional Reactive Programming (FRP) systems, maintaining global event order often introduces bottlenecks and degrades responsiveness. Czaplicki and Chong [19] addressed this in their design of Elm, an asynchronous Functional Reactive Programming language originally intended for graphical user interfaces. Elm introduces an asynchronous FRP model that allows for pipelined execution and allows events to be handled out of order through the async primitive. This construct enables long-running computations to occur without blocking the entire reactive system, thereby preserving responsiveness and enabling concurrent event processing.

Elm achieves this using a strong declarative abstraction to describe how values change over time, supporting event-driven behavior while avoiding unnecessary computations through its event-based update system. Its two-stage evaluation process, first handling functions and then signals, shows how reactivity can be built into a purely functional language in a predictable and efficient way. This is supported by a formal core model, FElm, which defines the underlying behavior of the language in a precise and reliable manner.

The reactive properties embedded in Elm's model have direct implications for swarm programming, particularly in supporting local event responsiveness and asynchronous consensus formation, which are essential in dynamic and partially connected networks. Embedding reactive semantics into swarm languages, as PiELo seeks to do, would allow each robot to quickly respond to changes in its environment or nearby agents without needing to stay in sync with the entire swarm, making it possible for coordinated behavior to emerge from local interactions.

Designing distributed systems that are robust to partial failures, operate under weak connectivity, and maintain consistency without centralized coordination is an essential challenge in distributed computing. A significant body of work has been dedicated to developing abstractions and programming models that enable coordination-free consistency while preserving system availability and scalability. Conflict-Free Replicated Data Types (CRDTs) have emerged as a principled approach to achieving strong eventual consistency in distributed systems, as it provides a way for multiple agents (replicas) to update shared data independently, without requiring coordination. CRDTs are designed data structures that ensure all replicas eventually reach the same final state, even if updates happen at different times or in different orders. This is especially valuable in distributed systems, where communication may be delayed, unreliable, or temporarily unavailable.

Preguiça et al. [20] present a comprehensive overview of CRDTs, outlining their core properties such as support for concurrent, unsynchronized updates at any replica, and deterministic convergence when replicas eventually receive the same set of updates. The authors discuss multiple types of CRDTs, including sets, registers, counters, and maps, each with distinct concurrency semantics such as add-wins, remove-wins, and last-writer-wins. State-based and operation-based are the two main flavors of CRDTs. In state-based CRDTs, each replica maintains its own local version of the data and occasionally shares the full state with others. When a replica receives a state from another, it merges it with its own using a join operation to ensure convergence, meaning that the order of merging does not matter and no data is lost. In operation-based CRDTs, rather than sharing full states, replicas exchange individual updates (operations), which must be commutative to guarantee that all replicas applying the same set of operations, regardless of order, reach the same state. Delta-based CRDTs are introduced in this article as an optimization of the state-based model. As previously discussed, the state-based CRDTs achieves synchronization by exchanging and merging entire states, which, despite being robust, becomes increasingly inefficient as the state grows. Delta-based CRDTs improve on this by propagating only small incremental changes, called deltas, rather than the full state. These deltas represent just the effect of a recent operation (or a batch of operations) and are significantly smaller in size. Each replica applies its local delta to its current state and shares the delta with other replicas. When other replicas receive the delta, they merge it into their own state using the same rules as standard CRDTs. This paper explores these synchronization models, offering an analysis of their trade-offs in terms of efficiency, convergence, and metadata overhead.

Building upon these ideas, Sanjuán et al. [21] propose an advanced synchronization framework known as Merkle-CRDTs, which combines CRDTs with Merkle Directed Acyclic Graphs (Merkle-DAGs) to support efficient and fully decentralized state propagation. In this model, CRDT updates are embedded directly as vertices within a Merkle-DAG, allowing replicas to encode and exchange causal histories in a cryptographically verifiable and structurally ordered manner. Rather than maintaining traditional version vectors, each replica synchronizes by identifying and transmitting only the missing branches of its DAG relative to its peers, significantly reducing communication overhead. A central contribution of the framework is the Merkle-Clock, a DAG-derived logical clock that replaces explicit timestamping by using the topology of the update graph to encode causal dependencies. The resulting system is highly modular, transport-agnostic, and well-suited for deployment in unstructured, peer-to-peer environments, such as robotic swarms, where communication is unreliable or intermittent. Furthermore, by decoupling the application logic from storage and transport layers, Merkle-CRDTs allow for flexible integration with protocols such as DHT or PubSub, and have been practically validated in distributed applications like collaborative editing and decentralized messaging. These properties show that Merkle-CRDTs are a promising abstraction

for swarm systems that require asynchronous, coordination-free consistency while maintaining robustness, scalability, and causality awareness.

Building upon the CRDT abstraction, Lasp [22] introduces a functional programming model for coordination-free distributed applications. Lasp composes CRDTs using higher-order functional operations, such as map, filter and fold, enabling deterministic and composable computations over distributed state. The model ensures strong eventual consistency by embedding monotonicity and convergence properties directly into the semantics of the language. Notably, Lasp permits applications to operate under weak or intermittent connectivity, delaying synchronization until connectivity is restored. The authors provide formal semantics for Lasp and demonstrate its practical applicability through an Erlang-based implementation built atop Riak Core, validated using a real-world use case from the SyncFree project.

Complementing the above approaches, Myter et al. [23] address the challenges of declaratively expressing and executing reactive data flow across distributed components. They observe that existing Distributed Reactive Programming (DRP) systems often rely on centralized coordination mechanisms, which hinder scalability and fault tolerance. In response, they propose QPROP, a novel propagation algorithm that ensures glitch-free, asynchronous dataflow in distributed dependency graphs without centralized control. Their work demonstrates how reactive semantics can be preserved in distributed architectures using the Spiders.js actor framework. The proposed system achieves eventual consistency while satisfying key reactivity properties such as elasticity and resilience.

Collectively, these studies contribute foundational abstractions and programming models for building distributed systems that minimize the need for synchronization, while preserving determinism and fault tolerance in the presence of concurrency and network failures. PiELo builds on this foundation by embedding a reactive primitive into a domain-specific language for swarm systems, while adopting a stigmergic consensus model inspired by Buzz embedding. This hybrid approach aims to enable agents in a swarm to respond expressively to local events and form robust consensus across distributed networks. By formalizing reactive and shared variables as language primitives, PiELo contributes a novel abstraction layer that advances the state of swarm programming towards more adaptive, scalable, and decentralized behaviors.

Chapter 3. Methodology

This chapter details the design and implementation of PiELo, covering the architecture of the PiELo virtual machine (VM), the treatment of closures (first-class function objects with captured state), the reactive programming model, the networking mechanism for sharing state among robots, memory management via garbage collection, native function interfacing and the compiler design.

3.1. Virtual Machine Architecture

PiELo programs execute on a custom VM built to run on resource-constrained swarm robots. The VM is implemented as a stack-based interpreter, meaning operands for computations are loaded onto an evaluation stack and most instructions implicitly operate on the top of this stack, rather than on named registers. This design was chosen primarily for its simplicity and portability, as stack-based bytecode is straightforward to generate and interpret [24]. At its core, the VM is a loop that fetches, decodes and executes *bytecode instructions*. A *program counter* (an index into the instruction list) tracks the current execution position, and it is progressed or modified according to each instruction's behavior (e.g. incremented for normal instructions or jumped for loop-control instructions). Our VM is implemented in C++ as a class encapsulating the instruction memory, the operand stack, a call stack for function invocation and a table of variables and their values.

3.1.1. Bytecode and Instruction Handling

PiELo source code is compiled into a bytecode format. Each instruction in this format consists of an opcode (operation code) and, optionally, operands (such as indices or literal values). For example, the VM defines opcodes for basic arithmetic, stack manipulation (such as push and pop), control flow (conditional jumps), functional calls, and special operations such as creating or returning from closures. *Labels* in the source (used for jumps/branching) are resolved to concrete bytecode addresses during compilation: the compiler assigns each label an address in the instruction array so that jump instructions can target them directly. The VM does not need to do any runtime lookup for labels - jumps are handled by setting the program counter to the target address.

3.1.2. Variable Class and Data Handling

All runtime data in PiELo is stored as a Variable.. Along with its value, each variable stores metadata to support reactivity and networking. Additionally, every Variable maintains a list of *dependent closures* - references to closures that rely on this Variable's value. This design enables the *propagation of change* for reactive programming (see Section 3.2). Variables can be created as needed during execution (e.g when a **var** declaration is executed) and are stored in an environment (a global variable map or local context for closures).

The VM uses the Variable table or environment to resolve variable names to their values at runtime. Through the Variable class, PiELo is able to uniformly handle primitive data and references to complex objects (like closures or shared data structures) in a consistent way.

3.1.3. Closure Representation and Lifecycle

Closures are central to PiELo, enabling both first-class functions and the reactive data-flow capabilities of our language. In programming language terms, a *closure* is essentially a record pairing a block of code (a function body/scope) with an environment of captured Variables. Unlike a plain function which has no state, a closure carries "bindings" for any free variables (variables from the

surrounding scope) used in its code, allowing it to be invoked later even if the original scope has been exited [25].

3.1.3.1. Closure Lifecycle

A closure's lifecycle in PiELo can be described in phases:

1. **Definition**: When a function/expression is defined, a closure object is allocated and its code and environment are set up as described. At this point, the closure may be stored in a variable (if it's a named function) or passed as an argument. If it is associated with a specific output (such as a reactive expression assigned to a variable), it is also recorded as such after the closure returns.
2. **Invocation/Execution**: When a function is called, the VM creates a new closure which begins as a copy of its definition. The parameters are then bound to the arguments by populating the closure's local environment. The instructions of the closure's bytecode are then executed until a *RETURN* instruction is found. This produces a return value which is pushed onto the stack to be accessed by the function's caller. In a reactive scenario, the closure also updates its cached value as described in Section 3.1.3.4.
3. **Suspension:** After execution, there are instances where the closure may remain in memory awaiting future use. If it was a one-time call (such as a normal function that is not stored anywhere), the closure instance may be eligible for cleanup by the garbage collection system (see Section 3.4) unless references to it were saved. However, if it were the case that the closure is part of a reactive network, it says "alive" with its cached value and dependency links intact, essentially going into an idle state until triggered again. The closure exists in the runtime's closure list.
4. **Re-triggering:** In the case of reactive closures, any change in a dependency will cause the closure to be re-executed (see Section 3.2). This can happen many times during the program's run. Each time, the closure updates its output and, depending on the circumstances, possibly other states.
5. **Termination and Collection:** Eventually, a closure may no longer be needed (e.g., a variable holding a closure is set to a different value or goes out of scope) or if the entire program is shutting down. When a closure is not referenced by any variable or other closure environment, and is not in any dependents list (meaning nothing will trigger it), it becomes eligible for garbage collection. The runtime's garbage collector will reclaim the memory of such closures (removing them from the closure list and deleting the object) as described in Section 3.4. Until the collector runs, a defunct closure might linger, but by design it will not be triggered because it has no dependents to notify it.

To summarize, in PiELo closures operate as both first-class functions and as active data flow components: They are able to capture the lexical scope, register themselves for automatic change and carry cached results. This is key in order to support our reactive programming alongside imperative function calls.

3.1.3.2. Closure Templates and Instantiation

The VM handles function definitions in an initial pass before it begins executing code. When it finds a function definition, it produces a *closure template* - in essence a pointer to the compiled sequence of bytecode instructions for the function's body, along with relevant metadata such as the number and name of parameters and dependencies. This template can be thought of as analogous to a

function prototype. At runtime, when the program defines the function, a closure instance is created from the template.

3.1.3.3. Scope Handling and Environment Binding

While code is being executed, the VM maintains a local symbol table, which is a structure containing all of the variables in the current local scope. This symbol table makes up one of the four places variables can exist, along with the global symbol table, the table of shared variables (see Section 3.3), and the stack. When a closure is called, the current local symbol table is stored as part of the caller's data, and is replaced with the called closure's symbol table. Initially, a local symbol table is populated with any arguments given in the function call, but it can be added to by declaring local variables. This design means that closures can only access local variables defined in that closure.

3.1.3.4. Dependency Tracking and Caching in Closures

Every closure in PiELo keeps track of which variables it depends on. When defining a closure, programmers explicitly mark dependencies using an apostrophe, as shown in Fig. 1. In this case, variable *a* is a dependency of function *f*. This is where the dependency list that each Variable maintains is utilized. When a closure is called, the runtime registers the closure as a dependent of each Variable that the closure reads from, using the list of dependencies given in the closure's definition. This creates a directed dependency graph: variables point to closures that should be re-run if they change.

Additionally, closures can maintain a cached result. In reactivity, (where the closures end up representing a continuously updated computation), the closure stores the last computed value of its expression, which can be used to quickly retrieve the current value of a reactive expression. Recomputation would occur immediately after a dependency is updated, so this cached value is guaranteed to be up to date.

3.2. Reactive Programming Mechanism

One of PiELo's distinctive features is its built-in support for *reactive programming*, meaning that certain variables automatically update in response to changes in others without explicit reassignment in the code. In a reactive paradigm, relationships between values are maintained automatically by the runtime. For example, if **a** is defined to be **b + c** reactively, then whenever **b** or **c** changes, **a** is recalculated immediately, keeping the invariant **a = b + c** true at all times (similar to a spreadsheet cell formula or a dataflow graph). We achieve this by leveraging the closure and dependencies mechanisms described above.

3.2.1. Reactive Closures and Variables

In PiELo, any expression assigned to a variable using the normal **var** declaration or a **set** operation can become *reactive* if it depends on other variables. Dependencies are explicitly marked using an apostrophe ('), as shown in Fig. 4. The compiler and runtime handle such an assignment by creating an *implicit closure* for the expression. Instead of simply computing the expression once, PiELo treats the right-hand side as a function of some inputs, and it keeps that function around as a closure. The result of the closure is then tied to the left-hand side variable. As a concrete example, consider PiELo code, in which a, b, and c are previously defined to be global reactive variables:

```
1    (set b 5)
2    (set c 10)
3    (set a (+ b' c'))
```

Figure 4: Reactive programming in PiELo

Here, rather than immediately calculating **b** + **c** and storing 15 in **a** permanently, PiELo creates a closure for the expression **(+ b c)**, with dependencies of **b** and **c**. This closure will capture references to both variables in its environment, computing the arithmetic initially and storing it in **a**, all while registering itself as a dependent of **b** and **c**. **a** is now effectively a *derived value* that is automatically kept in sync with both **b** and **c**. Should either of these change, our reactive model triggers an update to **a** immediately.

3.2.2. Propagation of Change

The runtime system propagates changes through a directed graph of dependencies, which is essentially an instance of the observer pattern for variables and closures. Every Variable knows which closures depend on it. When a variable's value is changed (through an assignment or by a function effect), the VM performs the following steps to propagate the update:

1. **Mark Variable Change:** The variable's new value is stored in its Variable object. The variable may also record a timestamp or version number if needed (for networking consistency, see Section 3.3)
2. **Notify Dependents:** The runtime iterates over the list of dependent closures attached to that variable. Each dependent closure is scheduled for re-execution. In the current implementation, scheduling is immediate and synchronous: the VM simply invokes each dependent closure one by one as part of processing the assignment. (In future or more complex scenarios, one might enqueue updates to be processed in an event loop, but PiELo assumes quick computations and handles them on the fly for simplicity).
3. **Closure re-execution:** When a closure is triggered, it runs its bytecode exactly as it did initially. It will typically read the current values of its input variables (which have just been updated or marked) and compute a new result. Because of the dependency registration, as the closure reads its inputs, it is already correctly associated with them (the associations were made during the first execution). The closure may update its *output variable*. In our example, the closure will set **a**'s value to the new sum. Making this assignment in turn triggers its dependents (should there be any).
4. **Preventing Redundant Updates:** In the current design, each assignment triggers immediate recomputation, so if two variables that feed the same closure both change due to a single higher-level event, the closure might run twice. However, since PiELo is single-threaded, changes are processed sequentially, and intermediate computations will still lead to the correct final state. An optimization using a dirty flag could skip the second computation if the closure was already run and marked clean with the latest values, but this is a refinement. In practice, because each dependent is triggered in turn, by the time the second input's change is processed, the closure's output may already reflect the first input's change; updating again

with the second input's change simply brings it to the final correct value. Cyclic dependencies are disallowed, as they would result in an infinite loop, so the dependency graph is expected to be acyclic. The runtime can include simple checks to avoid infinite recursion in case the programmer accidentally creates a cycle (for example, by detecting if a closure triggers itself directly or indirectly and breaking the loop or issuing a warning).

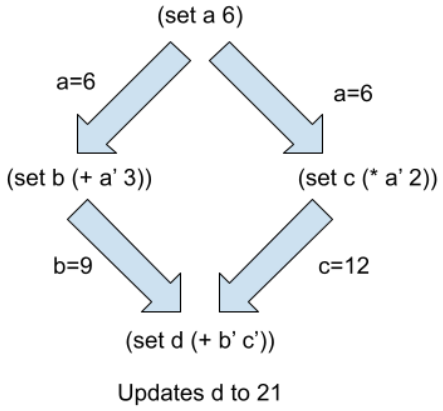


Figure 5: An example of change propagation in PiELo. All statements other than **(set a 6)** occurred earlier in the code. **b**, **c**, and **d** end the process with their cached values set correctly.

3.2.3. Guarantees and Semantics

The effect of PiELo's reactive system is that any *declared relationship* between variables is maintained automatically. The programmer does not have to write code to check and update dependent values; they simply declare the dependency once. PiELo provides these semantics in the context of multi-robot programs, which is particularly powerful: combined with the networking model, a change in one robot's state can automatically affect computations on another robot in near real-time, without explicit message handling in user code. This greatly simplifies coordinating behaviors in the swarm by raising the abstraction level to *shared reactive variables*.

3.3. Networking Model for Swarm Coordination

To extend reactive programming across a swarm of robots, we introduce a networking model that synchronizes *shared variables* between robots, as seen in Fig. 6. The key idea is that certain variables can be declared as global/shared, meaning their state is replicated across all robots in the network. When one robot updates a shared variable, that update is propagated to all other robots, causing their local copies of that variable to change and thus triggering any dependent closures on those robots.

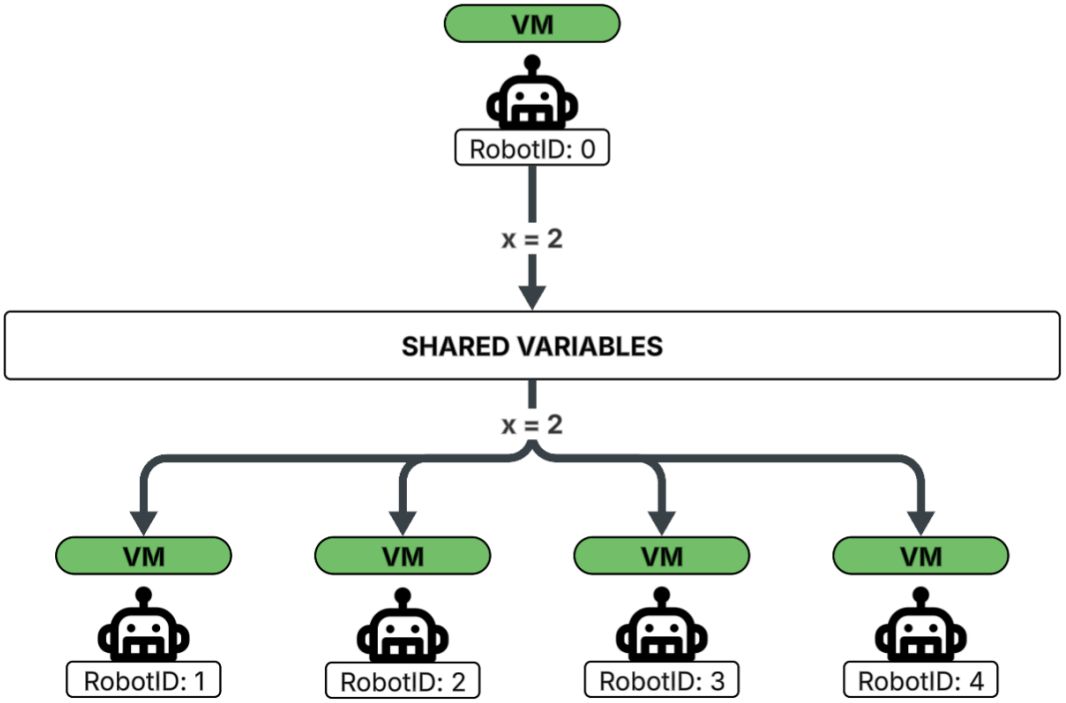


Figure 6: An example of shared variables broadcast across a swarm.

3.3.1. Shared Variable Synchronization

PiELo uses a *message broker* system to synchronize shared state across the swarm. One computer or designated robot acts as a server to which all PiELo-enabled robots are connected. The server handles the distribution of updates. The overall process works as follows:

1. At startup, each robot registers itself with the central server and initializes its set of shared variables (names and initial values).
2. When a robot's PiELo program executes an assignment to a variable that is marked as shared, the local VM not only updates the local value (and triggers local dependents), but also sends a network message to the server indicating the variable name (or ID), the new value, and an *update timestamp*. The timestamp is the robot's local time for consistency; the server can overwrite it with its own timestamp or sequence number upon receipt.
3. The central server receives the update message. It then blasts this update out to all connected robots (using a broadcast mechanism). This message contains the variable identifier and the new value, along with the timestamp. Because communication can have latency, PiELo uses the timestamp to perform *update validation*: each robot will only accept an incoming update for a shared variable if the timestamp is newer than the last seen update for that variable. This ensures that out-of-order network messages or delayed packets do not override more recent values. In essence, the system implements a *last-writer-wins* policy using timestamps.
4. Upon receiving a broadcast update, a robot's VM looks up the corresponding local Variable and compares the timestamp. If the incoming update is newer, the VM sets the variable's value to the received value (and updates the stored timestamp). This assignment is done through the same setter method described earlier, so it automatically triggers the reactive update propagation on that robot. In this way, a change originating on one robot cascades through any dependent computations on all other robots that share that data, keeping the swarm's state consistent.

This centralized approach is simple and ensures consistency at the cost of having a single point of communication. In practice, given the relatively small data (state variables) and the moderate size of swarms, a central coordinator is feasible. In addition, this centralized server could easily be swapped out in future work for a decentralized communication method. Communication was much easier for us to set up with the use of a router, but it could also be done with direct communication between the robots in the swarm.

3.3.2. Map Variables and Coordinated Behaviors

In addition to simple shared scalars, PiELo supports *map variables* (key–value stores) as a data structure for collective coordination. A map variable is essentially a dictionary keyed by robot ID that is shared across the network, allowing each robot to contribute or read entries. This is particularly useful for patterns like *barrier synchronization* or aggregation of sensor information. For instance, to implement a barrier (a point in the program where all robots must arrive before any can proceed), each robot can insert an entry into a shared map when it reaches the barrier. Suppose we have a shared map barrierMap where each robot writes some token value. As these updates propagate, every robot will eventually have a local map containing all entries from all robots. A robot can detect that the barrier is satisfied by checking if the map's size equals the number of robots (or if specific keys exist for all expected participants). This check can be done reactively as well – for example, a closure could be defined that monitors the map and proceeds when the condition is met. Once the barrier is passed, robots might clear their entries or the map can be reset for reuse.

3.3.3. Timing and Consistency

The use of timestamps gives a coherent *eventual consistency* model. All robots will converge to the same view of shared variables provided network messages get through. Currently, we do not implement reliability or retry beyond what the underlying transport provides, but since variables are often updated periodically or in response to events, occasional lost messages would be corrected by subsequent updates. If stronger guarantees are needed, an acknowledgment mechanism or state query could be added, but due to the scope of the project, we found the simple blast strategy effective. The central server can also act as a *time source* to avoid clock skew issues in timestamps: for instance, it can tag each update with a monotonically increasing sequence number or logical time.

3.4. Garbage Collection of Closures

Managing memory is critical in an interpreter running on potentially memory-limited robots. PiELo employs an *automatic garbage collection* mechanism to reclaim memory occupied by closures (and other dynamic objects) when they are no longer needed. The approach used is a classic *mark-and-sweep garbage collection* algorithm.

In garbage collection theory, an object is considered *garbage* (eligible for reclamation) if it is no longer reachable from any active part of the program. We define a set of *root references* from which "liveness" is determined. These roots include: (i) all global variables, (ii) any local variables in currently running closures (i.e., the call stack), (ii) any closures that are currently scheduled or in the middle of execution, and (iv) any native resources explicitly protected. The relationships among objects (variables hold references to closures in their dependents, closures hold references to variables in their environment, etc.) form a graph. The GC must traverse this graph to find what is still in use.

3.4.1. When Closures are Collected

A closure becomes a candidate for collection when nothing in the program needs it anymore. For example, if a closure was created to compute variable **a** and later **a** is assigned a new value by the program (breaking the link to the old closure), then that old closure is no longer referenced by **a**. If it was not referenced elsewhere, it would be garbage. Similarly, if a closure is a function that has completed execution and no other references to it exist (e.g., it wasn't stored in a variable or returned), it can be freed.

3.4.2. Mark-and-Sweep Strategy

The GC operates in two stages: mark and sweep: During the mark phase, the collector starts from the root set and traverses all references, marking each encountered object (closure, variable, etc.) as *reachable*. For each root variable, mark the variable and then mark any closure in its value (e.g., if a variable holds a closure reference), and for each closure, mark all variables in its environment and then recursively mark any closures reachable from those variables, and so on. Special care is taken to not follow the dependents lists during marking, since dependents link variables to closures (the graph of dependencies is essentially bipartite between variables and closures). We treat variables and closures uniformly in the mark algorithm – both can have references to each other. A closure might also have references to other closures (for instance, if a closure returns a function or stores a function in a variable captured by another closure). The mark routine sets a flag in each object it visits to indicate reachability.

In the sweep phase, the collector iterates over the global list of all allocated closures (the PiELo runtime maintains such a data structure for bookkeeping). Any closure not marked in the current GC cycle is deemed unreachable and is freed. "Freeing" involves removing it from the master list, deleting the object (invoking the C++ destructor to release memory), and also removing it from any ancillary structures (for example, if it was still erroneously in a dependents list of some variable, it would be removed to avoid dangling pointers). Similarly, if there is a list of allocated Variable objects (for variables without lexical scope like global variables or those dynamically created), the GC could free those that are no longer referenced; however, in PiELo, variables mostly exist in environment structures and are freed as part of closure collection or explicitly at program end. The sweep phase also clears the mark flags on surviving objects in preparation for the next cycle.

3.4.3. Integration with C++ Memory Management

Implementing a garbage collector in C++ requires careful integration with the language's manual memory model. PiELo's runtime objects (closures, etc.) are allocated on the heap (using the **new** keyword). The GC operates at the PiELo level by tracking PiELo objects, but ultimately it reclaims memory by calling **delete** on C++ objects. We chose to implement our own lightweight GC rather than use existing smart pointers or the Boehm conservative GC, because PiELo's object graph is well-defined and we can precisely manage it. Key considerations included avoiding moving objects (our GC is non-compacting, so pointers remain valid) and pausing execution at safe points for collection (e.g., after a certain number of instructions or when memory usage crosses a threshold). In practice, the collector can be invoked periodically or when an allocation fails. When triggered, PiELo VM's execution is paused, all robot operations momentarily halt in the VM, and the above mark-sweep routine runs to completion. The pause is usually very brief because the number of objects (closures) in typical programs is modest. After GC, execution resumes with freed memory back in the pool. This approach has proven sufficient for our use cases, ensuring that unused closures (for

instance, old reactive expressions that have been replaced) do not bog down the system or eat up memory over long runs.

3.5. Native Function Registration Interface

While PiELo is a high-level language, robotics requires interfacing with hardware (sensors, actuators) or performing specialized computations efficiently. To facilitate this, PiELo's VM allows registration of native functions (written in C or C++) that can be called from PiELo code as if they were ordinary PiELo functions. This mechanism enables extending the language with robot-specific primitives without altering the core VM for each new operation.

3.5.1. Registration Mechanism

The user can register a native function by providing its name and a function pointer (or functor) to the VM before running the PiELo program. For example, one might register a function **move_forward** that takes a distance and speed, implemented in C++ to command a robot's motors. Internally, the VM maintains a registry that maps a function name (string) to a function pointer (of a predefined signature). Each native function is expected to handle its own type checking or assume correct types based on how it's used in PiELo.

3.5.2. Calling Native Functions

When the compiler encounters a function call in the source, it must determine whether it is a user-defined (PiELo) function or a native function. Thankfully, this is made easy by consulting the environment that the compiler maintains which includes all defined functions. The programmer also has the ability to include a text file containing a list of all of the native functions which will be registered. The CALL instruction then checks the target of the call: if it is a native function, the VM calls it directly and pushes the result onto the stack; if it is a regular closure, the VM enters that closure's bytecode execution. Native calls thus do not create a new PiELo call frame or use the bytecode interpreter – they execute in C++ and return. This is efficient and also allows native code to perform tasks that PiELo bytecode would be too slow or incapable of (like interfacing with hardware drivers or performing heavy computations in optimized C++).

The integration is illustrated by a brief example. Suppose the PiELo program calls a function **distances = sense_distances()**. If **sense_distances** was registered as a native function that reads an array of distances from sensors, then when the CALL occurs, the VM will find the native function **sense_distances** and invoke it. To the PiELo program, it appears as if a normal function was executed, but under the hood it was a direct native operation.

3.5.3. Safety and Memory for Native Calls

Since native functions operate in C++ and can allocate PiELo objects (Variables, etc.), we ensure they cooperate with the GC. Any Variable created inside a native function that is returned or stored in a global must be properly registered so that it becomes part of the PiELo memory graph. If a native function creates temporary data and doesn't return it, that data should be freed or not allocated on the PiELo heap at all. We provide utility functions to native implementers to make this straightforward, such as factory functions to create a PiELo Variable of a given type.

From the user's perspective, the function registration system provides an FFI (Foreign Function Interface**)** style extension point. It allows wrapping existing robotics libraries or hardware

calls behind PiELo functions. This way, PiELo can remain minimalist (with only core language features implemented in the VM) while still being practical for real robotics tasks by leveraging native extensions. For instance, one can register functions for robot movement, sensor queries, inter-robot communication beyond shared variables (if needed), or computational routines (like path planning algorithms), and call them in PiELo scripts to coordinate complex behavior.

3.6. Compiler Design and Implementation

The PiELo language uses a syntax inspired by Lisp – a simple, parenthesized prefix notation for expressions and statements. The *compiler* translates PiELo source code (textual form) into the bytecode that the VM executes. The compilation process consists of *tokenization, parsing into an abstract syntax tree (AST)*, and code generation (emitting VM instructions).

3.6.1. Tokenization (Lexical Analysis)

The source code is first broken into a stream of tokens. Since PiELo's syntax is Lisp-like, the token types are typically: parentheses "(" and ")", symbols (alphanumeric identifiers for variables and function names), numbers (literals), string literals, and keywords (which are also symbols but recognized specially). The tokenizer scans the input string character by character, grouping characters into tokens.Whitespace and comments (if any syntax for comments exists) are skipped. For example, **(var a (+ b 1))** would be tokenized as: **(, var, a, (, +, b, 1,),)**. The outcome of this tokenization is then fed into the parser.

3.6.2. Parsing (Syntax Analysis)

We implemented a recursive descent parser that constructs an AST from the token stream. Given the prefix notation and the presence of parentheses to denote expression boundaries, the grammar is simple. Each expression is either a single token (atom) or a parenthesized list denoting a function call or special form.

3.6.3. AST to Bytecode Generation

Once the AST is built, the compiler traverses it to emit bytecode. The compiler maintains structures like a symbol table (for variable names and their scope levels) and a list of instructions being generated. For each node in the AST, code is generated according to the node type:

- For literal atoms (integers, floats), the compiler emits a PUSH_x (where x represents the type of the atom) instruction with the literal value. This will push the value onto the VM stack at runtime.
- For variable references, the compiler emits an instruction to load the variable (based on name) to load the current variable at runtime.
- For function call expressions (list where the first element is not recognized as a special keyword) the compiler handles them in a generic way: it assumes the first element of the list evaluates to a function/closure value, and the rest are arguments. It generates code to evaluate each argument in order (pushing their results on the stack), then generates code to evaluate the function expression (which could be a variable holding a closure or a lambda expression), and finally emits a CALL n instruction, where n is the number of arguments. At runtime this will cause the VM to call the function with those arguments.

3.6.4. Handling Special Constructs

The PiELo language defines a few special forms which the compiler must treat differently from normal function calls:

- **Conditionals (if):** Unlike a regular function, an if should evaluate *only* the branch that is needed (to support control flow and avoid side effects in the branch not taken). The compiler generates code for condition expression first. Then it emits a conditional jump: a placeholder JUMP_IF_FALSE instruction that will skip over the true branch code if the condition is false and generates an expression for the “true branch”. It performs a converse process for the other branch. The result is bytecode implementing the standard if-then-else logic.
- **Variable declaration (var):** This introduces a new variable. The compiler treats this as a declaration and initialization. It will ensure that a Variable entry is created in the current scope. It then compiles it into bytecode. However, instead of simply evaluating the expression and discarding it, the compiler must also assign it to the newly created variable. Due to our reactive design, if the initial expression has dependents, we want them to remain connected. The act of evaluating an expression that involves other variables will create a closure if needed. In practice, the compiler can emit code that signals the runtime to create a reactive closure. Our implementation handles this at runtime through the STORE_VAR instruction, when assigning to a brand new variable, detects if the value being stored is coming from evaluating an expression involving existing variables. It then encapsulates that expression into a closure automatically. In simpler terms, the initial evaluation of the variable will produce a value but also set up dependents, with STORE_VAR linking the closure to the variable name.
- **Function definition (fun):** The compiler handles this by creating a closure template for the body at compile time. It parses the body into an AST, then recursively compiles that body as a separate unit of code (with knowledge of the parameters as local variables). Instead of emitting code to execute the function body immediately, it emits code that will create a closure object at runtime.
- **Include:** This procedure allows the user to define a number of C functions which will be registered with the VM, and functions as described previously (Tokenizer simply registers the keyword and reads from the file path).

After handling all these, the final output of the compiler is a sequence of bytecode instructions ready to be loaded into the VM, as seen in Fig 7. The PiELo compiler is implemented in C++ as well, but it could be seen as a separate component or as part of the VM initialization.The resulting bytecode is quite compact and easily fits in the memory of small robots.

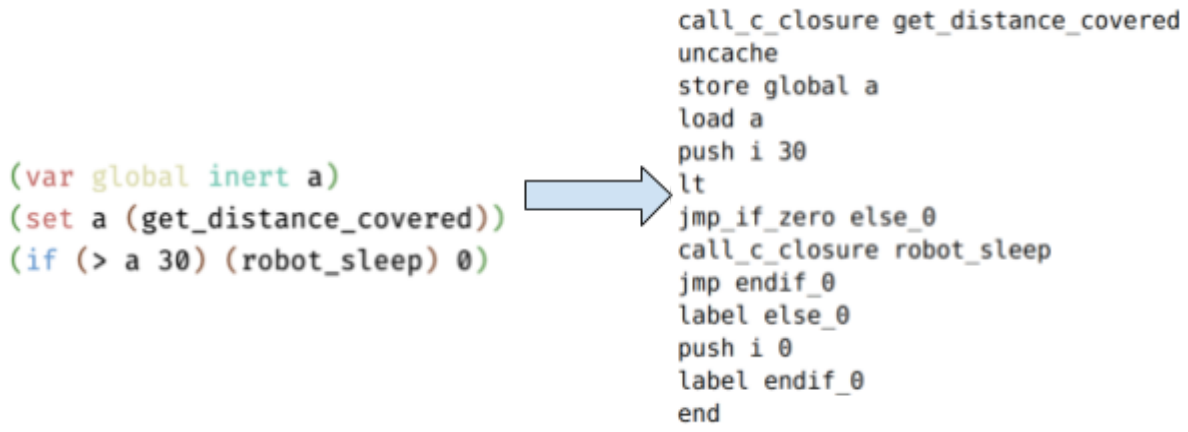


Figure 7: An example compilation of PiELo code. In this case, get_distance_covered is a C closure which interfaces with robot hardware.

3.6.5. Summary

Throughout this enterprise, we maintained a formal design approach, ensuring that each component of PiELo's implementation serves the goal of making swarm robotics programming more intuitive and less error-prone. By combining a stack-based VM, first-class closures with automatic reactivity, a shared memory model for inter-robot communication, automated memory management, and an extensible compiler, PiELo provides a coherent platform for declarative swarm programming. The next chapters will evaluate this implementation in practice and compare it to existing approaches. All the design choices described here were driven by the requirements identified in the introduction and related work, balancing expressiveness with runtime efficiency in the unique context of swarm robotics.

Chapter 4. Experimental Evaluation

4.1. Metrics

To measure the success of our implementation, we look at a few key aspects. One is the expressivity of the language. Our goal is to make programming for swarms easier, and one way to measure that is to look at the complexity of programs written to complete the same task. To evaluate the expressivity of PiELo, we conduct a comparative study using a simple barrier-test scenario implemented both in PiELo and in a traditional ROS-based system. We analyze the architectural complexity and count lines of code, focusing on conciseness, modularity, and clarity. These comparisons will help quantify how PiELo reduces cognitive and implementation overhead for developers.

We also define measurement criteria to evaluate reactivity and consensus. Though we are not focusing on performance optimizations, one measurement that we will take is the growth of added computation time as programs get more complex. The cost of reactivity is more behind-the-scenes computations that must take place on every variable update, as the virtual machine follows the tree of required updates.

The swarm's ability to achieve consensus can be measured through different metrics. The convergence time measures how long it takes for all agents in the swarm to agree on a common decision or reach a consensus. This measure can vary based on factors such as the swarm size, network topology and delayed communication in the network. Another important effective measure, particularly in noisy environments, is the consensus rate, which measures the percentage of successful consensus rounds. The consensus rate is the percentage of trials in which the swarm successfully reaches agreement within a defined time threshold. In this context, a consensus attempt refers to a single instance where agents in the swarm begin with deferring local values or opinions and attempt to align on a shared value. A successful consensus occurs when all agents converge to the same value within a set number of communication cycles or before a timeout. This metric helps assess how well the system handles uncertainty, message loss, or conflicting inputs.

Other important measures that we can use to evaluate consensus can be the accuracy of consensus, which evaluates how close the swarm's final decision is to an ideal or globally optimal solution,; and the robustness of the network, which measures how resilient the swarm's consensus is when some agents fail. While these metrics are critical for a comprehensive evaluation of a swarm programming language, we did not have the opportunity to explore them in this work and leave them for future investigation.

4.2. Setup

To evaluate how PiELo simplifies swarm programming, we designed an experiment based on a barrier-test scenario, a classical task in swarm robotics that requires both consensus and reactivity. The purpose of this setup is to demonstrate how easily and concise these complex behaviors can be expressed using PiELo, compared to a traditional framework like ROS, which requires significant infrastructure and implementation overhead.

The barrier-test involves a set of homogeneous agents moving forward at different speeds within a constrained arena. Each robot proceeds independently until it reaches a predefined checkpoint, a virtual or physical barrier marking a target distance from the starting point. The key requirement of the test is that: (i) No robot proceeds past the checkpoint until all robots have arrived, which requires consensus among the swarm. (ii) Once consensus is reached, all robots update their state and behavior simultaneously, highlighting the utility of reactivity in updating variables swarm-wide.

In PiELo, this entire behavior is coded declaratively using shared variables and reactive dependencies. When the last robot reaches the barrier, the shared consensus variable updates, and each robot automatically reacts to this update by setting its speed to a common value.

4.2.1. PiELo Implementation

The PiELo implementation of the barrier-test scenario consists of a single script written in the language itself, summing to just 22 lines of code.

```
(begin
(include functions.txt)
(var global inert barrierComplete)
(set barrierComplete 0)
(var global inert barrierSum)
(var map reactive barrier)

(fun reactive checkBarrier () (begin
    (var local inert barrierSum)
    (set barrierSum 0)
    (foreach inert (in i barrier') (set barrierSum (+ barrierSum i)))
    (if (> barrierSum 1) (set barrierComplete 1) 0)))

(var local inert total_distance)
(set total_distance 10)
(set distance_covered (get_distance_covered))
(set barrier (if (≥ distance_covered' total_distance) 1 0))
(set barrierChecker (checkBarrier))
(set speed (if (&& (≥ distance_covered' total_distance) (! barrierComplete)) 0 (* (+ (% robotID 5) 1) 2)))
(set leftWheelVelocity speed')
(set rightWheelVelocity speed')
(spin)
(fun reactive step () (set distance_covered (get_distance_covered))))
```

Figure 8: Barrier-Test implemented in PiELo.

The implementation defines:

1. A reactive local variable representing whether each robot has reached the checkpoint.
2. A shared swarm-wide consensus variable that reacts when all local flags are true.

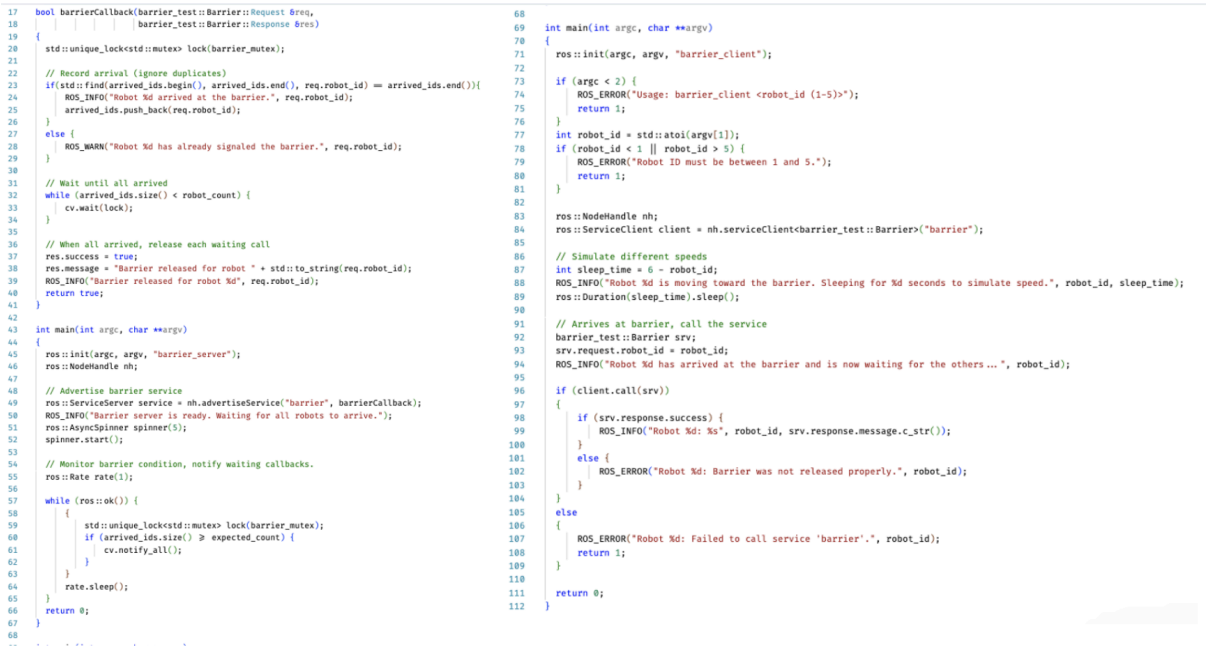
These high-level primitives eliminate the need for explicit state management, message passing, event-driven callbacks, or condition checking. PiELo's language runtime handles reactivity propagation and consensus formation automatically.

4.2.2. ROS Implementation

For comparison, we implemented the same barrier-test scenario using the widely-used Robot Operating System (ROS) framework. This implementation required:

1. Five separate files, including multiple ROS nodes and message definitions.
2. Custom publisher/subscriber logic to share each robot's state.
3. Explicit callbacks and condition checks to propagate updates across the swarm.
4. Manual implementation of logic to determine when consensus is reached.

In total, the ROS implementation spanned over 200 lines of code, highlighting the complexity and verbosity of expressing consensus and reactivity in traditional systems.



```cpp
bool barrierCallback(barrier_test::Barrier::Request &req,
                     barrier_test::Barrier::Response &res)
{
  std::unique_lock<std::mutex> lock(barrier_mutex);

  // Record arrival (ignore duplicates)
  if(std::find(arrived_ids.begin(), arrived_ids.end(), req.robot_id) == arrived_ids.end()){
      ROS_INFO("Robot %d arrived at the barrier.", req.robot_id);
      arrived_ids.push_back(req.robot_id);
  }
  else {
      ROS_WARN("Robot %d has already signaled the barrier.", req.robot_id);
  }

  // Wait until all arrived
  while (arrived_ids.size() < robot_count) {
      cv.wait(lock);
  }

  // When all arrived, release each waiting call
  res.success = true;
  res.message = "Barrier released for robot " + std::to_string(req.robot_id);
  ROS_INFO("Barrier released for robot %d", req.robot_id);
  return true;
}

int main(int argc, char **argv)
{
  ros::init(argc, argv, "barrier_server");
  ros::NodeHandle nh;

  // Advertise barrier service
  ros::ServiceServer service = nh.advertiseService("barrier", barrierCallback);
  ROS_INFO("Barrier server is ready. Waiting for all robots to arrive.");
  ros::AsyncSpinner spinner(5);
  spinner.start();

  // Monitor barrier condition, notify waiting callbacks.
  ros::Rate rate(1);

  while (ros::ok()) {
      {
          std::unique_lock<std::mutex> lock(barrier_mutex);
          if (arrived_ids.size() >= expected_count) {
              cv.notify_all();
          }
      }
      rate.sleep();
  }
  return 0;
}
```

```cpp
int main(int argc, char **argv)
{
  ros::init(argc, argv, "barrier_client");

  if (argc < 2) {
      ROS_ERROR("Usage: barrier_client <robot_id (1-5)>");
      return 1;
  }
  int robot_id = std::atoi(argv[1]);
  if (robot_id < 1 || robot_id > 5) {
      ROS_ERROR("Robot ID must be between 1 and 5.");
      return 1;
  }

  ros::NodeHandle nh;
  ros::ServiceClient client = nh.serviceClient<barrier_test::Barrier>("barrier");

  // Simulate different speeds
  int sleep_time = 6 - robot_id;
  ROS_INFO("Robot %d is moving toward the barrier. Sleeping for %d seconds to simulate speed.", robot_id, sleep_time);
  ros::Duration(sleep_time).sleep();

  // Arrives at barrier, call the service
  barrier_test::Barrier srv;
  srv.request.robot_id = robot_id;
  ROS_INFO("Robot %d has arrived at the barrier and is now waiting for the others ... ", robot_id);

  if (client.call(srv))
  {
      if (srv.response.success) {
          ROS_INFO("Robot %d: %s", robot_id, srv.response.message.c_str());
      }
      else {
          ROS_ERROR("Robot %d: Barrier was not released properly.", robot_id);
      }
  }
  else
  {
      ROS_ERROR("Robot %d: Failed to call service 'barrier'.", robot_id);
      return 1;
  }

  return 0;
}
```

Figure 9: Barrier-Test implemented in ROS.

4.2.3. Experiment

We validated the PiELo implementation of the barrier-test scenario in both simulated and real-world environments to assess the language's usability and expressivity. The goal of these experiments was not to evaluate performance under adverse conditions, but rather to examine how PiELo facilitates the development and deployment of reactive, consensus-based behaviors in swarm systems.

Simulation was conducted using the ARGoS3 multi-robot simulator, which provided a controlled yet realistic environment for observing swarm coordination. The PiELo script required no modifications to operate within the simulator, and all reactive and consensus behaviors emerged solely from the declarative code structure. This allowed us to verify that the intended logic, where agents stop at the barrier until all robots arrive and then proceed, was correctly triggered through PiELo's internal reactivity mechanisms, without the need for explicit synchronization logic.

To complement simulation, we deployed the same script on physical Khepera IV robots. These robots were equipped with onboard sensing and Wi-Fi communication capabilities and were deployed in a physical indoor arena with the Vicon motion capture system to provide real-time sensing positioning. This localization data was directly integrated into ARGoS3 using the same Vicon repository, enabling each robot to determine its position relative to the physical checkpoint (barrier). After we sshed in each robot and ran a remote endpoint, we were able to transmit the code and see the barrier experiment in action. The transition from simulation to hardware did not require rewriting or adapting the control logic, highlighting PiELo's portability across platforms. This demonstrates PiELo's potential to simplify the deployment process, which is a main challenge of usability in swarm programming.

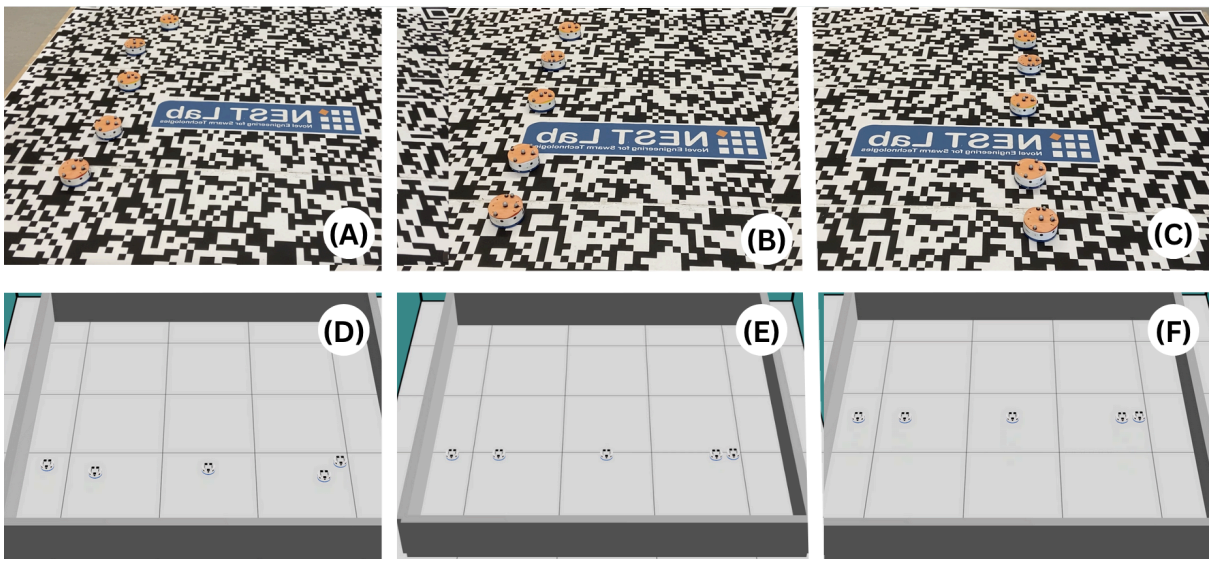


Figure 10: Comparison between simulation and real-world deployment of the barrier-test implemented in PiELo.
Simulation video: https://youtu.be/2cbGk5sQtqU
Real-world deployment video: https://youtu.be/b3TrQTtMfEQ

Testing across both simulated and physical environments reinforced the effectiveness of PiELo's abstractions. Consensus was consistently achieved, with agents stopping at the barrier and resuming coordinated movement solely through shared variables and reactive updates. Due to time constraints, this experiment was repeated only twice with swarms of relatively small size (3-5 robots).
This consistency across platforms, without the need for explicit coordination logic, provides evidence of PiELo's portability, usability and potential to reduce complexity in swarm programming.

4.3. Discussion

The results of our experimental evaluation highlight the core strengths and current limitations of PiELo as a domain-specific language for swarm programming. The expressiveness and conciseness of PiELo significantly stood out, especially when compared to the traditional ROS-based implementation of the barrier-test scenario. By embedding reactivity and consensus as core language features, PiELo drastically reduces the amount of code and infrastructure required to manage coordinated swarm behavior. This not only simplifies the development process but also lowers the barrier to entry for programmers who may not be familiar with low-level multi-agent synchronization or message-passing protocols.

Our findings demonstrate that PiELo enables developers to describe complex swarm behavior declaratively, leveraging shared variables and reactive dependencies without the need for manual state tracking or explicit communication protocols. This significantly improves code clarity and maintainability, as the same script is being executed by each robot, which is a critical factor in real-world deployments where debugging and iterating on swarm behaviors can be time-consuming and error-prone.

The successful replication of the barrier-test scenario in both simulation and hardware, without requiring changes to the PiELo script, underscores the language's portability and abstraction power. This seamless transition between platforms validates one of PiELo's goals, which is to simplify not only the development of swarm behaviors, but also their deployment across varied environments.

However, despite PiELo's potential to abstract swarm communication and coordination making swarm programming easy, PiELo's current limitations also must be acknowledged. PiELo's effectiveness in our study is observed under ideal assumptions, such as perfect communication and homogeneous agents. While this was necessary to isolate and assess the language's expressiveness, it does not reflect the challenges of real-world swarm systems, where delays, message loss, or partial failures are common occurrences. Additionally, the current implementation does not explore scalability limits or provide built-in support for fault tolerance, aspects which will be critical for future extensions of the language.

Despite these constraints, the simplicity and expressiveness of the PiELo DSL offer a compelling foundation for future development. By abstracting away many of the low-level mechanisms typically required in swarm coordination, PiELo enables programmers to easily write swarm behaviors and deploy them on real robots, paving the way for more complex applications that leverage broader data structures and logic.

Ultimately, PiELo's approach represents a promising shift in how we think about programming robot swarms, not as networks of independent systems requiring meticulous synchronization, but as a collective entity driven by shared reactive knowledge. The insights gained from this work lay the groundwork for expanding PiELo into a more robust, scalable, and fault-tolerant language, advancing the field toward more intuitive and accessible swarm programming frameworks.

Chapter 5. Conclusions

5.1. Summary

In this work, we introduced PiELo, a domain-specific language designed for programming robot swarms, with reactivity and consensus built in as core primitives. Our focus was on exploring the feasibility and expressiveness of these features, rather than optimizing for performance or robustness. To keep the scope manageable, we assumed ideal conditions, such as perfect communication between robots, which allowed us to concentrate on what makes PiELo unique.

We defined success based on how PiELo simplifies swarm programming and makes complex behaviors such as consensus and reactivity easier to express. To evaluate this, we compared a barrier-test scenario implemented in PiELo against a traditional ROS implementation. In this scenario, robots drive at different speeds until reaching a shared checkpoint. Once all have arrived, they continue forward at the same speed, requiring swarm-wide coordination. Our PiELo solution accomplished this in just 22 lines of code, while the ROS version spanned over 200 lines across five files. This clearly shows how PiELo reduces the complexity of expressing swarm behaviors, as reactivity automatically keeps variables up to date across the swarm, and consensus is handled seamlessly through shared variables.

While this version of PiELo doesn’t yet address real-world challenges such as network delays, agent failures, or large-scale swarms, it proves that these high-level concepts can be successfully embedded into a language. By giving developers powerful tools out of the box, PiELo makes swarm programming more intuitive, concise, and maintainable, setting the stage for future development in more realistic settings.

5.2. Future Work

While PiELo demonstrates the feasibility of integrating reactivity and consensus as core language primitives for swarm programming, there are several areas that need future exploration.

One key limitation of our current work is the assumption of perfect communication among agents. Future versions of PiELo should explore how reactivity and consensus perform under more realistic network conditions, such as packet loss, latency, and unstable connectivity. This would involve developing communication methods that can handle network issues, while preserving the correctness and synchronization guarantees provided by the language.

Another area of interest is scalability. Although our current evaluation focuses on relatively small swarms, it's important to assess how PiELo behaves as the number of agents increases. This includes understanding the impact on computation and memory overhead, as well as the performance of the virtual machine driving reactivity and consensus.

Another area for future work is the robustness of consensus mechanisms. Introducing controlled agent failures or message corruption could help us evaluate how PiELo handles faults, and could lead to the development of built-in fault-tolerant primitives for consensus.

From a language design perspective, future versions of PiELo may benefit from integrating a broader set of data types, which can enable the support for heterogeneous agents and different swarm behaviors. These enhancements would increase the expressiveness and versatility of the language, making it applicable to a wider range of swarm robotics applications.

Through these future directions, PiELo can evolve into a more comprehensive and reliable tool for programming autonomous robot swarms in real-world scenarios.

5.3. Lessons Learned

Undertaking the design and implementation of PiELo taught us many valuable lessons, be those technical, methodological or life lessons.

Firstly, we discovered how project goals can evolve over time. At the outset, we defined our success largely in terms of completion of features - implementing every component we thought was required for a language. This led us to getting distracted by ideas that, at the end of the day, were nothing more than syntactic sugar. By the end, however, we came to value robust core functionality over these peripheral extensions. This shift in perspective came from a deeper understanding of the problem: It is remarkably easy to misjudge the relative difficulty and importance of features envisioned at the start of a project, and even more so to overestimate how critical early ambitions of a project can be once a system begins to take shape.

A concrete example of this is mentioned in the Future Work Section above (5.2). In early development, we were working on a tagging system to support heterogeneity in the swarm. We design a data structure and basic runtime support for robots to be tagged (e.g., flying, gripper, etc.) that could gate tag-exclusive functions. Over time, however, our development efforts were mostly dominated by the complexity of the reactive closures and the creation of the Virtual Machine itself. As we prioritized a stable event-driven core, the tagging subsystem quickly fell by the wayside.

This reprioritization was not a failure in itself but a strategic reorientation: we chose to focus on building a viable reactive-programming platform that met our primary goal - making it easy to program event-driven behaviors - rather than dispersing our effort across multiple ambitious features that don't necessarily help us achieve this goal. Reflecting on this tradeoff led us to our second, and arguably more important lesson: to be captivated by the problems themselves, not their solutions. It is rather easy to be blinded by a specific idea to solve a problem, but no solution is without its shortcomings. One must be poised enough to realise this when striving to make new contributions to their fields.

References

[1] Bonabeau, Dorigo, Theraulaz. Swarm Intelligence: From Natural to Artificial Systems. Oxford University Press, 1999, p. 7.
[2] G. Beni, “From Swarm Intelligence to Swarm Robotics,” Swarm Robotics, vol. 3342, pp. 1–9, 2005.
[3] Oliveira, M., Pinheiro, D., Macedo, M. *et al.* Uncovering the social interaction network in swarm intelligence algorithms. *Appl Netw Sci* **5**, 24 (2020). https://doi.org/10.1007/s41109-020-00260-8
[4] C. Pinciroli, “Intro to Swarm Intelligence” slides 12-14.
[5] Carlo Pinciroli, Vito Trianni, Rehan O'Grady, Giovanni Pini, Arne Brutschy, Manuele Brambilla, Nithin Mathews, Eliseo Ferrante, Gianni Di Caro, Frederick Ducatelle, Mauro Birattari, Luca Maria Gambardella, Marco Dorigo. 2012. ARGoS: a Modular, Parallel, Multi-Engine Simulator for Multi-Robot Systems. Swarm Intelligence, volume 6, number 4, pages 271-295. Springer, Berlin, Germany.
[6] C. Pinciroli, “Swarm Programming” slides 3-8
[7] Danilo Pianini, Mirko Viroli, and Jacob Beal. 2015. Protelis: practical aggregate programming. In Proceedings of the 30th Annual ACM Symposium on Applied Computing (SAC '15). Association for Computing Machinery, New York, NY, USA, 1846–1853. https://doi.org/10.1145/2695664.2695913
[8] Karamcheti, S., Nair, S., Chen, A. S., Kollar, T., Finn, C., Sadigh, D., & Liang, P. (2023). Language-Driven Representation Learning for Robotics. https://arxiv.org/abs/2302.12766
[9] O’Grady, R., Christensen, A.L., Dorigo, M. (2012). SWARMORPH: Morphogenesis with Self-Assembling Robots. In: Doursat, R., Sayama, H., Michel, O. (eds) Morphogenetic Engineering. Understanding Complex Systems. Springer, Berlin, Heidelberg.
https://doi.org/10.1007/978-3-642-33902-8_2
[10] J. McLurkin, J. Smith, J. Frankel, D. Sotkowitz, D. Blau, and B. Schmidt, “Speaking Swarmish : Human-Robot Interface Design for Large Swarms of Autonomous Mobile Robots,” in AAAI Spring Symposium: To Boldly Go Where No Human-Robot Team Has Gone Before. AAAI, 2006, pp. 3–6.
[11] J. Bachrach, J. Beal, and J. McLurkin, “Composable continuous-space programs for robotic swarms,” *Neural Computing and Applications*, vol. 19, no. 6, pp. 825–847, May 2010.
[12] M. P. Ashley-Rollman, P. Lee, S. C. Goldstein, P. Pillai, and J. D. Campbell, “A language for large ensembles of independently executing nodes,” in *Logic Programming*, ser. LNCS 5649, P. M. Hill and D. S. Warren, Eds., vol. 5649 LNCS. Springer Berlin Heidelberg, 2009, pp. 265–280.
[13] Dedousis, D., & Kalogeraki, V. (2018). A Framework for Programming a Swarm of UAVs. *Proceedings of the 11th PErvasive Technologies Related to Assistive Environments Conference*, 5–12. https://doi.org/10.1145/3197768.3197772
[14] Pinciroli, C., & Beltrame, G. (2016). Buzz: An extensible programming language for heterogeneous swarm robotics. *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3794–3800. https://doi.org/10.1109/IROS.2016.7759558
[15] St-Onge, D., Varadharajan, V., Li, G., Svogor, I., & Beltrame, G. (2017). *ROS and Buzz: Consensus-based behaviors for heterogeneous teams*. https://doi.org/10.48550/arXiv.1710.08843
[16] Moussa, M., & Beltrame, G. (2020). On the robustness of consensus-based behaviors for robot swarms. *Swarm Intelligence*, *14*(3), 205–231. https://doi.org/10.1007/s11721-020-00183-1
[17] Chen, W., Liu, J., & Guo, H. (2020). Achieving Robust and Efficient Consensus for Large-Scale Drone Swarm. *IEEE Transactions on Vehicular Technology*, *69*(12), 15867–15879. https://doi.org/10.1109/TVT.2020.3036833
[18] Li, X., Gao, M., Kang, Z., Sun, H., Liu, Y., Yao, C., & Zhang, A. (2023). Collaborative search and rescue based on swarm of H-MASSs using consensus theory. *Ocean Engineering*, *278*, 114426. https://doi.org/10.1016/j.oceaneng.2023.114426

[19] Czaplicki, E., and Chong, S. (2013). Asynchronous Functional Reactive Programming for GUIs. In 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2013). ACM, 411–422. https://doi.org/10.1145/2499370.2462161
[20] Preguiça, N., Baquero, C., Shapiro, M. (2018) “Conflict-Free Replicated Data Types (CRDTs).” *arXiv.Org*, 16 May 2018, arxiv.org/abs/1805.06358.
[21] Sanjuan, H., Poyhtari, S., Teixeira, P., Psaras, I. (2020). “Merkle-CRDTs: Merkle-Dags Meet CRDTs.” *arXiv.Org*, 27 Apr. 2020, arxiv.org/abs/2004.00107.
[22] Meiklejohn, C., & Van Roy, P. (2015). Lasp: A language for distributed, coordination-free programming. *Proceedings of the 17th International Symposium on Principles and Practice of Declarative Programming*, 184–195. https://doi.org/10.1145/2790449.2790525
[23] Myter, F., Scholliers, C., De Meuter, W. (2019) “Distributed Reactive Programming for Reactive Distributed Systems.” arXiv.Org, 1 Feb. 2019, arxiv.org/abs/1902.00524.
[24] M. Schinz, EPFL, www.epfl.ch/labs/lamp/wp-content/uploads/2019/01/acc14_08_interpreters-vms.pdf. Accessed 1 May 2025.
[25] Sussman, G. J., & Steele Jr., G. L. (1998). An interpreter for extended lambda calculus. *Higher Order Symbolic Computation*, *11*(4), 405–439. https://doi.org/10.1023/a:1010035624696